# When does A* save work?

## T01 checkpoint: question, contracts and frozen protocol

Elnur Haqqverdiyev  Rajab Iskandarli

UFAZ, M1 Data Science and AI
Advanced Data Structures and Algorithms, Fall 2026

Checkpoint

# The question is not “is A* faster?”

**Baseline:** Dijkstra with a binary heap and lazy deletion.
**Comparison:** A* with the same heap and loop, plus a heuristic.

A* can reduce *expanded vertices*, but it pays for:

- one heuristic evaluation per generated vertex;
- larger heap keys and more memory traffic;
- extra work that is wasted if the heuristic is uninformative.

**Research question**

How do graph structure and heuristic strength change expanded vertices and runtime of A* versus Dijkstra, on unit-cost grids and a real road graph?

**Deliverable:** a rule for when the expansion saving pays for the guidance cost.

# Predictions to test

| | prediction |
|---|---|
| H1 | open grid: A* saves many expansions and time |
| H2 | random obstacles: saves, but less than open |
| H3 | clustered/rooms: weaker; rooms weaker than random |
| H4 | maze: no expansion saving, A* may be slower |
| H5 | long queries benefit more than short |
| H6 | real ALT: strong at K=8; K=1 may not help; diminishing at K=16 |
| H7 | `larger_g` reduces median expansions on open grid |

These are written in `HYPOTHESES.md` before the main run; a negative result is acceptable.

# Data choice

**Grids as controls**

- open, random 20%, clustered 30%
- rooms with doors
- perfect maze
- 128, 256, 512, 1024 square
- 4-neighbour, unit cost

Why: isolate one structural property at a time.

**Real graph**

- SNAP `roadNet-CA`, California roads
- 1,957,027 vertices, 2,760,388 undirected edges after preparation
- largest connected component
- unit hop weight (SNAP has no lengths/travel times)

Why: sparse, planar-like, and exactly the setting where A* is usually recommended.

# ADT contracts

## Graph: CSR

`offsets[n+1]`, `targets[m]`, `weights[m]`
memory $O(n+m)$; degree $O(1)$; neighbours $O(\deg u)$

## Binary heap

push/pop $O(\log n)$; top $O(1)$
duplicates allowed; stale entries discarded lazily
tie-break: $(f, -g, id)$ or $(f, id)$

## Search

one loop; Dijkstra $= h \equiv 0$
A* $= f = g + h(v)$; stop when target settled
consistent $h \Rightarrow$ no reopening
O(1) scratch reset by generation stamp

## Heuristics

grids: Manhattan, admissible + consistent
real: ALT $\max_L |d(L, v) - d(L, t)|$
landmark BFS precomputed outside timing

# Correctness before benchmarking

- hand-computed 5x5 wall grid (distance 8)
- six-vertex weighted graph vs Floyd–Warshall
- all five grid families vs independent BFS
- ALT admissibility/consistency for every free vertex
- path validation: contiguous edges and exact weight sum
- `expanded`, `stale_popped`, `pushed` identities

## Main-run guard

Every benchmark query is checked against BFS. A distance mismatch aborts the run. Timeouts are recorded, never silently dropped.

## Counter definitions

expanded = settled pop (target included)
pushed = source + decreases
stale = old entry or already closed

# Frozen protocol

| experiment | queries | timed runs |
|---|---|---|
| grids (20 graphs) | 300 | 2,700 |
| real graph | 30 | 900 |
| total | 330 | 3,600 |

3 grid repeats; 5 real repeats. One untimed warm-up per treatment.

- paired same query, alternating order
- median per query; raw repeats kept
- bootstrap 95% CI + Wilcoxon
- 30 s/search, 4 GiB/process
- one CPU via `taskset`
- seed 20261030

Protocol fixed at this checkpoint; changes require a deviation note.

# Prototype status and next steps

**Working now**

- C++20 CSR graph, binary heap, Dijkstra/A*
- five grid generators
- Manhattan and ALT heuristics
- BFS/Floyd–Warshall oracles
- correctness test suite
- raw-CSV benchmark runner

**Next**

- pilot run to confirm the time budget
- main grid and real runs
- paired analysis, CIs and figures
- defence: answer the question, including negative results

The implementation is the same loop for both treatments; only the heuristic changes.